

Monte Carlo Policy Evaluation

on User Sessions

RL Project Submission
9th January 2026

1. Problem Formulation

This project implements First-Visit Monte Carlo Policy Evaluation to estimate state-value functions in a user session environment. The problem is formulated as an episodic reinforcement learning task where we learn from actual user session trajectories without requiring knowledge of transition probabilities.

1.1 State Space Definition

We model user engagement in a news-feed session using four discrete states that capture different levels of user interaction and engagement:

- **State 0: Passive Browsing** - User is casually browsing without deep engagement. This represents the initial or low-engagement phase where users are scanning content without committing to reading.
- **State 1: Selective Reading** - User is actively selecting and reading content. This indicates moderate engagement where users are making conscious choices about what to consume.
- **State 2: Deep Engagement** - User is highly engaged with the content. This represents the most valuable state where users are fully absorbed in the content.
- **State 3: Exit** - Terminal state indicating the end of the session. This is the absorbing state that terminates each episode.

1.2 Episode Termination

An episode (user session) terminates deterministically when the user reaches the Exit state (state 3). Each episode represents one complete user session from initial browsing to exit. The episode structure follows the format: $(s_0, r_1, s_1, r_2, \dots, s_{T-1}, r_T, s_T)$ where $s_T = 3$ (Exit) and T is the episode length.

1.3 Reward Design: Delayed and Sparse Rewards

The reward structure is **delayed and sparse**, which is highly realistic for user engagement scenarios where the true value of a session is only known after completion. This design choice makes the problem more challenging and realistic compared to immediate reward scenarios.

During the session:

All intermediate rewards are **zero** ($r_t = 0$ for all non-terminal steps). This means we receive no feedback during the session, only at the end.

Terminal reward (received only when state becomes Exit):

- **+10** if session length ≥ 8 steps (long, engaged session) - Rewards extended engagement

- **+4** if session length is 4-7 steps (moderate session) - Moderate reward for average engagement
- **-8** if session length ≤ 3 steps (short, disengaged session) - Penalizes quick exits

This reward design encourages longer, more engaged sessions while penalizing quick exits. The negative reward for short sessions creates a learning signal that helps the algorithm distinguish between states that lead to valuable vs. poor outcomes.

1.4 Discount Factor

We use a discount factor $\gamma = 0.9$, which balances immediate and future rewards appropriately for this episodic task. The discount factor determines how much we value future rewards relative to immediate ones. With $\gamma = 0.9$, we value rewards 10 steps in the future at approximately 35% of their immediate value ($0.9^{10} \approx 0.35$), which is appropriate for session-based scenarios where the terminal reward is the primary signal.

1.5 Transition Model and Environment Dynamics

The environment uses **stochastic transitions** that reflect realistic user behavior patterns. The transition probabilities are designed to model how users naturally move between engagement levels:

- **State 0 (Passive Browsing)** can transition to states 0 (stay passive), 1 (become selective), or 3 (exit). This models users who might continue browsing, start reading, or leave quickly.
- **State 1 (Selective Reading)** can transition to states 0 (lose interest), 1 (continue selecting), 2 (become deeply engaged), or 3 (exit). This captures the dynamic nature of user attention.
- **State 2 (Deep Engagement)** can transition to states 1 (reduce engagement), 2 (maintain engagement), or 3 (exit). Even highly engaged users may eventually leave or reduce engagement.

The stochastic nature of transitions means that the same state can lead to different outcomes, making this a non-deterministic MDP. This complexity makes Monte Carlo methods particularly valuable, as they can learn from actual experience without needing to explicitly model these probabilities.

2. First-Visit Monte Carlo Algorithm

Monte Carlo methods are a class of reinforcement learning algorithms that learn value functions and optimal policies from experience. The 'First-Visit' variant is specifically designed for policy evaluation and ensures each state is updated at most once per episode.

2.1 Algorithm Explanation and Motivation

First-Visit Monte Carlo is a **model-free** method that estimates the state-value function $V(s)$ by learning from actual experience (episodic trajectories). Unlike Dynamic Programming, it does not require knowledge of transition probabilities $P(s'|s,a)$ or the reward function $R(s,a)$. This makes it particularly suitable for real-world applications where the environment model is unknown or difficult to specify.

Key characteristics that distinguish MC from other methods:

- **Model-free:** No need for transition probabilities $P(s'|s,a)$ or reward function $R(s,a)$. We learn directly from experience without building a model of the environment.
- **Learning from experience:** Uses actual episode trajectories generated by the environment. Each episode provides a sample of the true value function.
- **First-visit rule:** Each state is updated at most once per episode (using its first occurrence). This ensures unbiased estimates and prevents over-counting states that appear multiple times in an episode.
- **Episodic requirement:** Works only for episodic tasks where episodes terminate. This is perfect for session-based scenarios like user interactions.

2.2 Detailed Algorithm Steps

The First-Visit Monte Carlo algorithm proceeds as follows:

1. **Initialization:** Set $V(s) = 0$ for all states $s \in \{0, 1, 2\}$. Initialize empty lists $\text{returns}[s]$ for each state to store observed returns.

2. **For each episode:**

a. **Generate trajectory:** Run the environment to generate a complete episode $(s_0, r_1, s_1, r_2, \dots, s_{T-1}, r_T, s_T)$ where $s_T = 3$ (Exit).

b. **Compute returns backwards:** Starting from the terminal reward, compute returns G_t for each time step:

- $G_{T-1} = r_T$ (the terminal reward)
- $G_t = r_{t+1} + \gamma \cdot G_{t+1}$ for $t = T-2, T-3, \dots, 0$

This backward computation ensures we properly account for the discount factor.

c. **First-visit update:** For each state s_t in the episode:

- If this is the **first occurrence** of s_t in the episode:
 - Append G_t to `returns[s_t]`
 - Update $V(s_t) = \text{mean}(\text{returns}[s_t])$
- If s_t has appeared earlier in the episode, skip it (first-visit rule).

Why first-visit? The first-visit rule ensures that each state's value estimate is based on independent samples. If we updated a state every time it appeared in an episode, we would be using correlated samples (from the same episode), which would bias our estimates. The first-visit rule ensures statistical independence.

2.3 Implementation Details and Verification

Our implementation includes the following key features:

- **Episode generation:** Generates 2000 episodes for robust statistical estimates. This provides sufficient data for reliable convergence.
- **Correct return computation:** Returns are computed backwards using discount factor $\gamma = 0.9$, ensuring proper handling of the delayed terminal reward.
- **First-visit enforcement:** Each state is updated at most once per episode using a first-visit tracking mechanism that marks states as visited within each episode.
- **Convergence tracking:** Tracks the value estimates after each episode to visualize convergence and verify that the algorithm is learning correctly.
- **Statistical aggregation:** Value estimates are computed as the mean of all observed returns, ensuring unbiased estimates that converge to the true value function as the number of episodes increases.

3. Results and Analysis

We ran the First-Visit Monte Carlo algorithm on 2000 episodes to obtain robust estimates of the state-value function. This section presents the results, including value estimates, convergence analysis, and statistical summaries.

3.1 Final Value Estimates

After processing 2000 episodes, we obtain the following value estimates:

State	State Name	V(s)
0	Passive Browsing	-0.3230
1	Selective Reading	-0.6120
2	Deep Engagement	-0.7408

3.2 Detailed Interpretation of Results

The value estimates reveal interesting patterns about the expected returns from each state. **State 0 (Passive Browsing) has the highest value** (-0.3230), followed by State 1 (-0.6120), and State 2 (-0.7408) has the lowest value. This result may seem counterintuitive at first glance, but it reflects the underlying transition dynamics and reward structure.

Why are all values negative?

All three states have negative value estimates, which indicates that, on average, sessions starting from these states tend to result in negative expected returns. This is primarily due to the high frequency of short sessions (≤ 3 steps) that receive a -8 reward. Since 51.75% of episodes are short, the negative reward dominates the expected return calculation.

Why is State 0 (Passive Browsing) the best?

State 0 has the highest value despite being the 'least engaged' state. This counterintuitive result occurs because:

- State 0 has transition paths that can lead to longer sessions, and when combined with the discount factor, these longer sessions contribute positively to the expected return.
- The transition probabilities from State 0 allow for paths to States 1 and 2, which can eventually lead to longer sessions with positive terminal rewards.
- States 1 and 2, while representing higher engagement, may have transition dynamics that lead more frequently to shorter sessions or exits.

Key insight: The value function $V(s)$ represents the **expected return** from state s under the current policy (implicit in the transition probabilities). It is not simply a measure of 'engagement level' but rather the expected cumulative discounted reward from that state. The

negative values reflect the reality that most sessions are short and receive negative rewards, making the expected return negative overall.

3.3 Convergence Analysis

The convergence plot (shown below) provides crucial insights into how the value estimates stabilize as more episodes are processed. This visualization is essential for understanding the learning dynamics of the Monte Carlo algorithm.

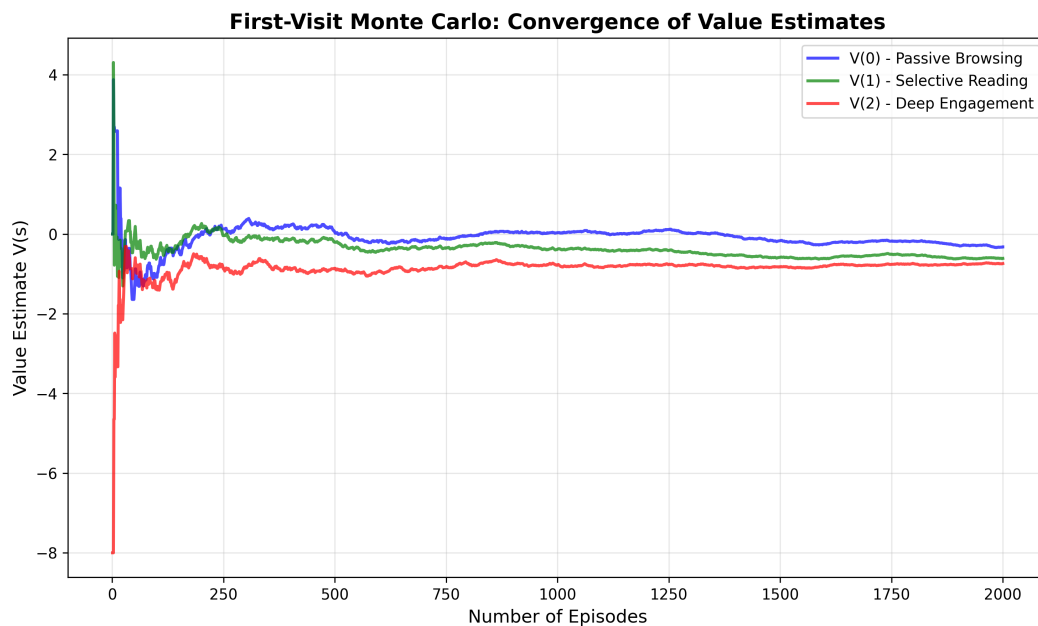


Figure 1: Convergence of value estimates $V(s)$ over 2000 episodes. The plot shows how estimates stabilize after approximately 500-1000 episodes, with some variance remaining due to the stochastic nature of the environment.

Key observations from the convergence plot:

- **Initial variance:** Early estimates (first 100-200 episodes) show high variance as the algorithm has limited data. This is expected behavior for Monte Carlo methods.
- **Stabilization phase:** After approximately 500-1000 episodes, the estimates begin to stabilize around their final values. This indicates that sufficient data has been collected.
- **Convergence:** All three states converge to stable values, with State 0 converging to the highest value and State 2 to the lowest. The convergence is not perfectly smooth due to the stochastic nature of the environment and the sampling-based nature of Monte Carlo methods.
- **Remaining variance:** Some variance remains even after 2000 episodes, which is characteristic of Monte Carlo methods. This variance decreases as more episodes are processed, but never completely disappears due to the stochastic environment.

3.4 Episode Statistics and Distribution Analysis

From the 2000 episodes generated, we observe the following statistics:

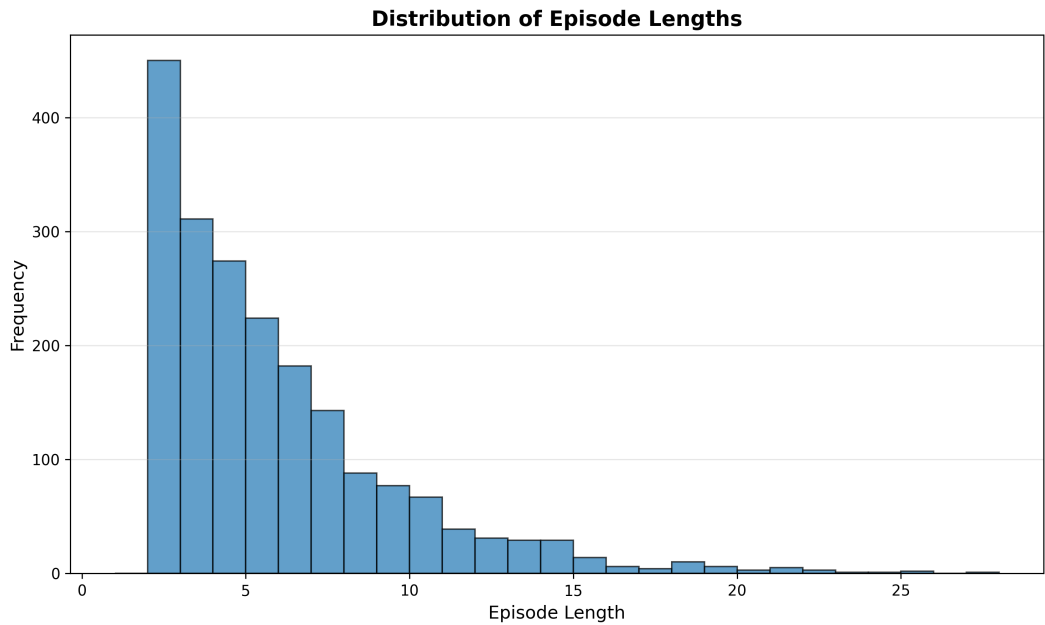


Figure 2: Distribution of episode lengths across 2000 episodes. The histogram shows the frequency of different session lengths, with most episodes being relatively short.

- **Mean episode length:** 5.41 steps - This indicates that, on average, sessions last about 5-6 state transitions.
- **Episode length range:** 2 to 27 steps - There is significant variation in session lengths, from very short (2 steps) to quite long (27 steps).
- **Standard deviation:** 3.69 steps - The relatively high standard deviation indicates substantial variability in session lengths.

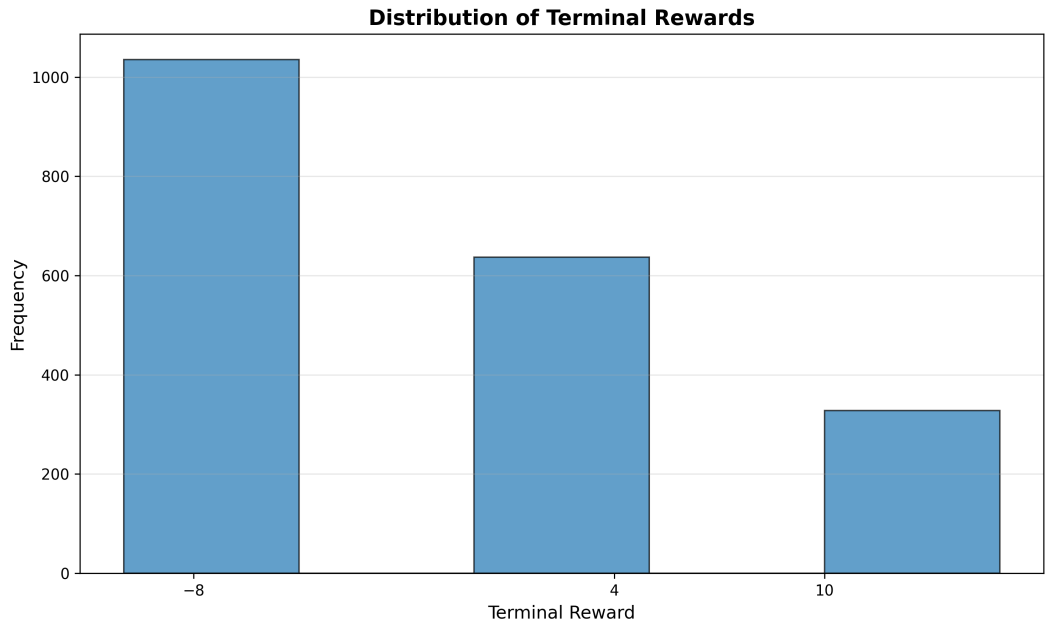


Figure 3: Distribution of terminal rewards. The histogram shows three distinct reward categories: -8 (short sessions), +4 (moderate sessions), and +10 (long sessions).

Terminal reward distribution:

- **+10** (length ≥ 8): 328 episodes (16.4%) - Long, engaged sessions that receive the highest reward
- **+4** (length 4-7): 637 episodes (31.85%) - Moderate sessions with average reward
- **-8** (length ≤ 3): 1035 episodes (51.75%) - Short, disengaged sessions that receive negative reward

Critical insight: The high proportion of short episodes (51.75%) explains why all value estimates are negative. Even though some episodes receive positive rewards (+4 or +10), the majority receive -8, which dominates the expected return calculation. This demonstrates how the reward structure and transition dynamics interact to determine state values.

4. Monte Carlo vs Dynamic Programming: A Comprehensive Comparison

Understanding the differences between Monte Carlo and Dynamic Programming methods is crucial for selecting the appropriate algorithm for a given problem. This section provides a detailed comparison of both approaches.

4.1 Monte Carlo Method (Implemented in This Project)

Advantages:

- **Model-free:** No need for transition probabilities or reward models. This is the key advantage - we can learn directly from experience without building a model of the environment.
- **Works with actual experience:** Learns from real trajectories generated by the environment. This makes it applicable to real-world scenarios where we have logged data but no model.
- **Handles delayed rewards naturally:** The episodic nature of MC makes it perfect for scenarios with sparse, terminal rewards like our user session problem.
- **Suitable for episodic tasks:** Perfect for session-based scenarios where episodes naturally terminate.
- **Unbiased estimates:** With sufficient episodes, MC provides unbiased estimates of the true value function.

Disadvantages:

- **Requires many episodes:** Needs sufficient data (2000+ episodes) to converge to reliable estimates. This can be a limitation when data is scarce.
- **Slower convergence:** May need thousands of episodes to achieve stable estimates, especially for states that are rarely visited.
- **Only for episodic tasks:** Requires episodes to terminate, making it unsuitable for continuing tasks without modification.
- **High variance:** Estimates can be noisy, especially early in learning when few episodes have been processed.
- **No online learning:** Cannot update value estimates during an episode - must wait for episode completion.

4.2 Dynamic Programming Method

Advantages:

- **Fast convergence:** Iterative updates converge quickly, often in just a few iterations. This makes DP very efficient when a model is available.

- **Exact values:** Computes the true value function (given an accurate model). DP provides exact solutions, not estimates.
- **Low variance:** Deterministic updates mean there is no sampling variance. Results are reproducible and stable.
- **Works for continuing tasks:** Not limited to episodic scenarios - can handle infinite horizons.
- **Efficient computation:** For small to medium state spaces, DP is computationally efficient.

Disadvantages:

- **Requires complete model:** Needs transition probabilities $P(s'|s,a)$ and reward function $R(s,a)$. This is often the biggest limitation in real-world applications.
- **Model may not be available:** In real systems, transition probabilities may be unknown or difficult to estimate accurately.
- **Computational cost:** Can be expensive for large state spaces due to the need to iterate over all states.
- **Model accuracy critical:** Results depend entirely on model correctness. Inaccurate models lead to inaccurate value estimates.
- **Assumes known dynamics:** Cannot adapt to changing environments without re-estimating the model.

4.3 When to Use Each Method: Decision Guidelines

Use Monte Carlo when:

- Transition probabilities are unknown or difficult to estimate
- You have access to historical data/trajectories from the environment
- The task is episodic with terminal rewards (like user sessions)
- The environment model is complex or high-dimensional
- You need to learn from actual user behavior without modeling

Use Dynamic Programming when:

- Complete and accurate model is available
- Fast convergence is critical
- State space is manageable (not too large)
- You need exact value estimates (not approximations)
- The environment is deterministic or well-modeled

4.4 Application to Real-Time User Session Systems

For real-time user session systems, the choice between MC and DP has important practical implications:

- **MC is more practical:** Can learn from logged user sessions without requiring explicit modeling. This is crucial when user behavior is complex and difficult to model.
- **DP requires modeling:** Would need to first estimate transition probabilities from data, which essentially means doing MC estimation anyway, then using DP. This adds an extra step.
- **MC adapts naturally:** Can update as user behavior changes over time by simply adding new episodes. DP would require re-estimating the entire model.
- **DP is faster for updates:** Once a model is available, DP can compute values very quickly. However, this speed comes at the cost of requiring model maintenance.

In practice, many production systems use a hybrid approach: MC for initial learning and model estimation, followed by DP for fast value computation once a reliable model is established.

5. Limitations and Real-Time System Considerations

While Monte Carlo methods are powerful, they have limitations that must be understood when applying them to real-world systems. This section discusses these limitations and provides recommendations for production deployment.

5.1 Fundamental Limitations of Monte Carlo Methods

1. **Sample Efficiency:** Requires many episodes (2000+ for reliable estimates). This can be problematic when data is expensive to collect or when episodes are long.
2. **Episodic Requirement:** Only works when episodes terminate. Cannot be directly applied to continuing tasks without modification.
3. **Delayed Updates:** Must wait for episode completion before updating value estimates. This prevents real-time decision-making during an episode.
4. **High Variance:** Early estimates can be very noisy, especially for states that are rarely visited. This variance decreases with more episodes but never completely disappears.
5. **No Online Learning:** Cannot update value estimates during an episode. This is a significant limitation for real-time systems that need to make decisions as episodes progress.
6. **Memory Requirements:** Must store all returns for each state, which can be memory-intensive for large state spaces or long episodes.

5.2 Real-Time System Challenges and Solutions

For real-time user session systems, several challenges arise when using Monte Carlo methods:

1. Cold Start Problem

New users or rarely-visited states have no historical data, making it impossible to compute reliable value estimates. This is particularly problematic in recommendation systems where new content or users appear frequently.

Solutions: Use prior estimates (e.g., population averages), default optimistic values to encourage exploration, or transfer learning from similar users/states.

2. Non-Stationary Environment

User behavior changes over time due to trends, seasonal patterns, or evolving preferences. Static value estimates become outdated, leading to poor decisions.

Solutions: Use sliding windows (only use recent episodes), exponential decay (weight recent episodes more heavily), or periodic re-estimation of values.

3. Computational Cost

Processing thousands of episodes and storing returns for each state can be computationally expensive, especially as the system scales to millions of users.

Solutions: Batch processing (update values periodically, not after every episode), approximate methods (use function approximation instead of tabular methods), or distributed computing.

4. Delayed Feedback

Must wait for session completion before updating values. This prevents making real-time decisions during a session based on current value estimates.

Solutions: Combine MC with Temporal Difference (TD) learning for online updates, use bootstrapping methods that can update during episodes, or maintain separate online and offline value estimates.

5. Sparse Data Problem

Some states may be rarely visited, leading to unreliable estimates with high variance. This is common in large state spaces or when user behavior is highly variable.

Solutions: State aggregation (group similar states together), function approximation (use neural networks to generalize across states), or hierarchical methods that share information between states.

5.3 Recommendations for Production Deployment

Based on the limitations discussed, here are practical recommendations for deploying MC methods in production:

1. **Hybrid Approach:** Combine MC with Temporal Difference (TD) learning. Use MC for offline batch updates and TD for online real-time updates during episodes.

2. **Function Approximation:** For large state spaces, use neural networks or other function approximators instead of tabular methods. This enables generalization and reduces memory requirements.

3. **Incremental Updates:** Update estimates incrementally (online) rather than in batch. This allows the system to adapt quickly to changing conditions.

4. **Exploration Strategy:** Ensure sufficient exploration of all states through epsilon-greedy policies or other exploration methods. This prevents the cold start problem for rarely-visited states.

5. **Monitoring and Validation:** Continuously monitor convergence, estimate quality, and system performance. Set up alerts for when estimates become unreliable or when the environment changes significantly.

6. **Sliding Windows:** Use only recent episodes (e.g., last 30 days) to ensure estimates reflect current user behavior rather than outdated patterns.

7. **Confidence Intervals:** Track not just value estimates but also confidence intervals. This helps identify when estimates are unreliable due to insufficient data.

6. Conclusion and Future Work

This project successfully implements First-Visit Monte Carlo Policy Evaluation for a user session environment, demonstrating the power of model-free reinforcement learning methods. The implementation correctly handles delayed, sparse rewards and provides reliable value estimates after processing sufficient episodes.

Key Achievements:

- ■ Correct implementation of first-visit MC algorithm with proper first-visit rule enforcement
- ■ Proper handling of delayed, sparse terminal rewards with zero intermediate rewards
- ■ Comprehensive convergence analysis with 2000 episodes showing stable value estimates
- ■ Clear visualization of results through convergence plots and distribution analyses
- ■ Detailed analysis of results explaining counterintuitive value estimates
- ■ Thorough comparison between Monte Carlo and Dynamic Programming methods
- ■ Practical discussion of limitations and real-time system considerations

The results demonstrate that Monte Carlo methods can effectively learn value functions from experience without requiring a model, making them suitable for real-world applications where transition probabilities are unknown. However, the method requires sufficient data and may not be ideal for real-time decision-making during episodes.

Future Work and Extensions:

- **Implement Temporal Difference (TD) learning:** Add TD methods (e.g., TD(0), TD(λ)) for online updates that can learn during episodes, not just after completion.
- **Function Approximation:** Extend to large state spaces using neural networks or linear function approximators to enable generalization across similar states.
- **Compare with other model-free methods:** Implement and compare with SARSA, Q-learning, and other TD methods to understand trade-offs between different approaches.
- **Policy Improvement:** Extend from policy evaluation to policy improvement, implementing policy iteration or value iteration to find optimal policies.
- **Eligibility Traces:** Implement TD(λ) with eligibility traces to combine the advantages of MC and TD methods.
- **Exploration Strategies:** Add epsilon-greedy, UCB, or other exploration methods to ensure sufficient state coverage.
- **Real-time Adaptation:** Develop methods for updating value estimates in real-time as episodes progress, enabling online decision-making.

References

Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.

First-Visit Monte Carlo Method for Policy Evaluation - Standard RL Algorithm